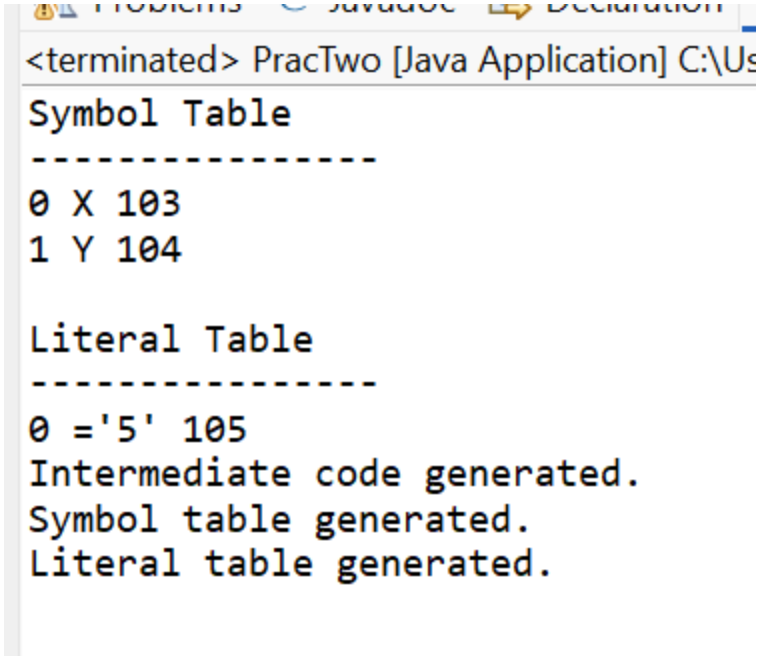
ASSIGNMENT NO. 2

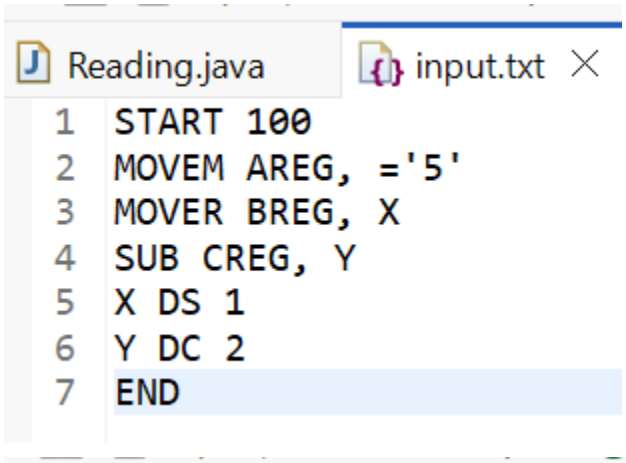
```
package Hi;
import java.io.*;
import java.util.*;
public class PracTwo {
  static int LC = 0;
  public static void main(String args[]) {
     HashMap<String, String> opcode = new HashMap<>();
     HashMap<String, String> reg = new HashMap<>();
     LinkedHashMap<String, Integer> symbol = new LinkedHashMap<>();
     LinkedHashMap<String, Integer> literal = new LinkedHashMap<>();
     opcode.put("ADD", "IS,01");
     opcode.put("SUB", "IS,02");
     opcode.put("MULT", "IS,03");
     opcode.put("MOVER", "IS,04");
     opcode.put("MOVEM", "IS,05");
     opcode.put("COMP", "IS,06");
     opcode.put("BC", "IS,07");
     opcode.put("DIV", "IS,08");
     opcode.put("READ", "IS,09");
     opcode.put("PRINT", "IS,10");
     opcode.put("STOP", "IS,00");
     reg.put("AREG", "RG,01");
     reg.put("BREG", "RG,02");
     reg.put("CREG", "RG,03");
     reg.put("DREG", "RG,04");
     try{
        BufferedReader br=new BufferedReader(new FileReader("input.txt"));
        BufferedWriter bw=new BufferedWriter(new FileWriter("op.txt"));

        String line;
        while ((line = br.readLine()) != null) {
           line = line.trim();
           if (line.isEmpty())
              continue;
           line = line.replace(",", "");
           String t[] = line.split("\\s+");
           // START
           if (t[0].equalsIgnoreCase("START")) {
              LC = Integer.parseInt(t[1]);
              bw.write("(AD,01) (C," + LC + ")");
              bw.newLine();
           }
           // IMPERATIVE STATEMENT
           else if (opcode.containsKey(t[0].toUpperCase())) {
              bw.write(LC + " (" +opcode.get(t[0].toUpperCase()) + ")");
              // REGISTER
```

```
    if (t.length > 1 &&reg.containsKey(t[1].toUpperCase())) {

        bw.write(" (" + reg.get(t[1].toUpperCase()) + ")");
    }
    // OPERAND
    if (t.length > 2) {
        String operand = t[2];
        // LITERAL
        if (operand.startsWith("=")) {
            if (!literal.containsKey(operand)) {
                literal.put(operand, -1);
            }
            int index =new ArrayList<>(literal.keySet()).indexOf(operand);

            bw.write(" (L," + index + ")");
        }
        // SYMBOL
        else {
            if (!symbol.containsKey(operand)) {
                symbol.put(operand, -1);
            }
            int index =new ArrayList<>(symbol.keySet()).indexOf(operand);

            bw.write(" (S," + index + ")");
        }
    }
    bw.newLine();
    LC++;
}
// DS
else if (t.length > 2 &&t[1].equalsIgnoreCase("DS")) {

    symbol.put(t[0], LC);
    bw.write(LC + " (DL,01) (C," + t[2] + ")");

    bw.newLine();
    LC += Integer.parseInt(t[2]);
}
// DC
else if (t.length > 2 &&t[1].equalsIgnoreCase("DC")) {

    symbol.put(t[0], LC);
    bw.write(LC +" (DL,02) (C," +t[2] + ")");
    bw.newLine();
    LC++;
}
// END
else if (t[0].equalsIgnoreCase("END")) {
    bw.write("(AD,02)");
```

```
                bw.newLine();
                // ASSIGN ADDRESSES TO LITERALS
                for (String lit : literal.keySet()) {
                    literal.put(lit, LC);
                    String value =lit.replace("='", "").replace("'", "");

                    bw.write(LC +" (DL,02) (C," + value + ")");
                    bw.newLine();
                    LC++;
                }
            }
        }
        br.close();
        bw.close();
        // SYMBOL TABLE
        System.out.println("Symbol Table");
        System.out.println("----------------");
        int i = 0;
        for (String s : symbol.keySet()) {
            System.out.println(i + " " +s + " " +symbol.get(s));
            i++;
        }
        // LITERAL TABLE
        System.out.println();
        System.out.println("Literal Table");
        System.out.println("----------------");
        i = 0;
        for (String l : literal.keySet()) {
            System.out.println(i + " " +l + " " + literal.get(l));
            i++;
        }
        System.out.println("Intermediate code generated.");
        System.out.println("Symbol table generated.");
        System.out.println("Literal table generated.");
    }
    catch (Exception e) {
        System.out.println(e);
    }
  }
}
```

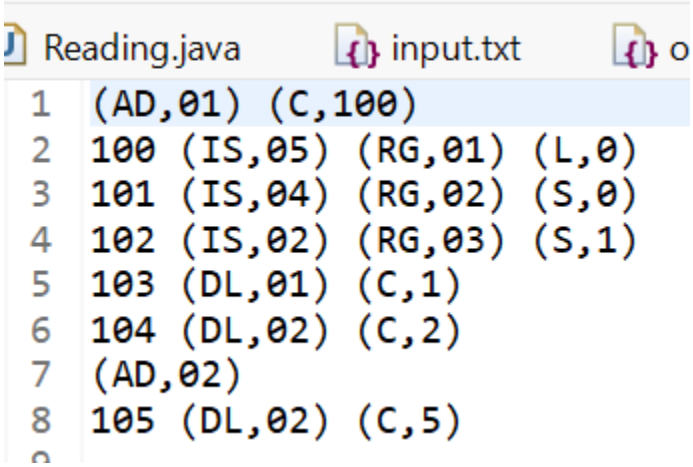


```
<terminated> PracTwo [Java Application] C:\Us
Symbol Table
----------------
0 X 103
1 Y 104

Literal Table
----------------
0 ='5' 105
Intermediate code generated.
Symbol table generated.
Literal table generated.
```



Reading.java | input.txt

```
1  START 100
2  MOVEM AREG, ='5'
3  MOVER BREG, X
4  SUB CREG, Y
5  X DS 1
6  Y DC 2
7  END
```



Reading.java | input.txt | o

```
1  (AD,01) (C,100)
2  100 (IS,05) (RG,01) (L,0)
3  101 (IS,04) (RG,02) (S,0)
4  102 (IS,02) (RG,03) (S,1)
5  103 (DL,01) (C,1)
6  104 (DL,02) (C,2)
7  (AD,02)
8  105 (DL,02) (C,5)
```